Universidad Distrital Francisco José de Caldas

Computer Engineering Program — School of Engineering

Systems Analysis & Design — Semester 2026-I

Eng. Carlos Andrés Sierra, M.Sc.

Community Security Alert System (CSAS) Technical Report

Workshops No. 1 & No. 2 — Systems Analysis and Design

Felipe Jose Garzon Herrera *Code: 20251020132*

Juan Esteban Quintero Gordillo *Code: 20251020137*

Henry Samuel Garrido Medina *Code: 20251020125*

Gabriel Mateo Cusba Marin *Code: 20251020128*

Bogotá D.C., Colombia — March 2026

Submitted in partial fulfillment of the requirements for the Systems Analysis & Design course, Semester 2026-I.

0 Abstract

Campus security at Universidad Distrital Francisco José de Caldas is impaired by three compounding problems identified through systematic primary data collection: a persistent under-reporting culture attributable to the high friction of current reporting channels, a spatial and temporal concentration of incidents during evening and night hours in specific campus zones, and the absence of a centralized real-time incident management tool that forces security staff to rely on phone calls and walk-up reports, resulting in average response delays of approximately 11.4 minutes. This report presents the Community Security Alert System (CSAS), a crowd-sourced mobile and web platform that addresses these gaps through a microservices-based, event-driven architecture integrating a hybrid AI-and-human verification pipeline, geofenced alert dispatch with zone-weighted and time-weighted priority logic, and a multi-channel notification engine. The proposed design targets a reduction of emergency response time to under 5 minutes, an increase of incident recording completeness from 41% to above 85%, and a community adoption rate of at least 60%.

Keywords: campus security, crowd-sourced incident reporting, microservices architecture, real-time notifications, geofenced alerts, systems analysis, systems design, Universidad Distrital.

Contents

List of Figures

List of Tables

1 Introduction

Campus safety is a persistent challenge for universities in urban Latin American environments. At Universidad Distrital Francisco José de Caldas, a public university located in Bogotá D.C., the absence of a centralized digital reporting tool means that security incidents are communicated through informal channels — phone calls, WhatsApp groups, and walk-up reports to security guards — introducing delays, data loss, and a systemic inability to detect spatial or temporal patterns. The average time from incident occurrence to security team notification has been measured at approximately 11.4 minutes, a figure that represents a significant risk during fast-developing emergencies such as assaults or medical events.

Several platforms have attempted to address similar challenges in the Bogotá metropolitan area. The Policía Nacional's Vigila App integrates directly with law enforcement but relies on the user placing a phone call, producing slow response cycles. Neighborhood WhatsApp networks achieve high adoption rates due to their familiarity but operate without any verification mechanism, systematically generating false alarms and panic. SIURE UD, the university's own internal incident reporting system, provides an existing institutional process but operates exclusively through email and phone, offering no mobile interface or real-time notification capability [1]. These precedents establish a clear gap: no existing platform combines low-friction mobile reporting, real-time geofenced notification, and verified alert quality in an institutional campus context.

The Community Security Alert System (CSAS) is proposed to fill this gap. It is designed as a crowd-sourced platform in which students, faculty, and staff act as active sensors for community safety, submitting geolocated incident reports through a mobile interface while a hybrid AI-and-human verification pipeline ensures signal quality before alerts are broadcast to the affected campus community. This report integrates two phases of the project: Workshop No. 1, which presents the empirical systems analysis conducted to characterize the problem, and Workshop No. 2, which presents the formal systems design derived from those findings.

2 Literature Review

The theoretical foundation of this project draws from established systems thinking and engineering frameworks. Checkland's soft systems methodology [2] provides the conceptual basis for treating campus security as a sociotechnical system in which human behavior, institutional processes, and technology interact in complex, non-linear ways. Meadows [3] offers the analytical vocabulary for understanding the feedback dynamics that govern adoption and alert quality in a crowd-sourced platform: the reinforcing loops that amplify participation and the balancing loops that must be calibrated to prevent alert fatigue. The ISO/IEC/IEEE 15288:2015 standard [4] provides the life-cycle process framework within which the CSAS analysis and design activities are situated. Creswell and Creswell [5] inform the mixed-methods data collection strategy employed in Workshop No. 1, combining structured interviews, surveys, direct observation, document analysis, benchmarking, and environmental monitoring into a multi-method design

that strengthens the validity of findings through methodological triangulation. From the software architecture perspective, Bass, Clements, and Kazman [7] ground the design decisions related to modularity, scalability, and separation of concerns that motivate the microservices decomposition. Dragoni et al. [8] provide a contemporary treatment of microservices as an architectural style, validating its suitability for systems with heterogeneous load profiles across subsystems — precisely the case in the CSAS, where verification, notification, and analytics exhibit independent scaling requirements.

3 Objectives

3.1 General Objective

To design a community-driven, real-time incident reporting and alert system for the Universidad Distrital Francisco José de Caldas campus that demonstrably reduces emergency response times and improves incident recording completeness relative to the current state.

3.2 Specific Objectives

1. To characterize the current campus security posture through primary data collection, identifying key gaps, behavioral parameters, and system boundaries (Workshop No. 1).

2. To translate the empirical findings of the systems analysis into a formal set of functional and non-functional requirements explicitly traceable to observed problems.

3. To design a microservices-based architecture that addresses the identified gaps through modular, independently scalable components.

4. To define a hybrid AI-and-human verification pipeline that balances alert latency with alert signal quality.

5. To specify a technical implementation strategy — including technology stack, implementation phases, and key performance indicators — as a foundation for future development.

4 Scope

The CSAS scope is defined by a boundary that separates the platform's direct responsibilities from its operational environment. Table 1 formalizes this boundary. The system covers the campus of Universidad Distrital Francisco José de Caldas and its immediate surrounding streets, and serves six stakeholder groups: students (primary reporters and alert recipients), professors and administrative staff (secondary reporters), campus security staff (verifiers and response coordinators), local authorities (recipients of escalated verified incidents), system administrators (platform maintainers), and university management (consumers of analytics and security reports).

Table 1. System Boundary Definition — CSAS

Inside the system (scope)	**Outside the system (environment)**
Incident reporting and management platform	University physical infrastructure
Crowd-sourced verification logic	Local law enforcement (Police / EMS)
Real-time push notification engine	Public telecommunications and ISP networks
User and incident databases	Non-registered external visitors
Emergency coordination dashboard	Campus security physical hardware (CCTV, etc.)
Analytics and heatmap generation	University academic or administrative systems

5 Assumptions

The analysis and design documented in this report rest on three critical assumptions that have not yet been validated under real operating conditions and that represent the primary behavioral and technical risks for the implementation phase.

Willingness to report. It is assumed that students and staff will submit incident reports if the reporting process is made sufficiently simple and low-friction. Workshop No. 1 survey data provides directional support: all participants who had personally experienced a campus incident stated they would use the application. However, this preference expressed under survey conditions has not been confirmed in real operational settings. This assumption is designated for validation through user acceptance testing in Phase 3.

Verification latency tolerance. The design assumes that the verification mechanism can process incoming reports and produce a verified alert within a 45-second latency budget for standard-severity incidents. The validity of this assumption depends entirely on implementation-specific decisions — including AI model performance, confirmation threshold configuration, and cloud infrastructure capacity — that fall outside the scope of this design document and must be validated through load testing in Phase 4.

Mobile connectivity coverage. The system assumes reliable mobile network and WiFi connectivity across all campus zones. The direct observation phase of Workshop No. 1 did not include a systematic assessment of signal coverage. Dead zones in low-connectivity areas could materially degrade alert delivery guarantees and represent a prerequisite risk for infrastructure planning.

6 Limitations

Several limitations constrain the reach of the conclusions drawn in this report. These are acknowledged not as weaknesses that invalidate the work, but as boundaries that define the scope of the evidence base and establish a research agenda for subsequent phases.

The student survey conducted in Workshop No. 1 ($n = 20$) is too small for statistical generalization to the full university population. The 35% incident prevalence estimate is a meaningful directional indicator but cannot be treated as a representative rate without a larger, stratified sample. The direct observation phase covered only four campus zones across eight sessions; indoor corridors, cafeterias, and transit areas between buildings were not observed, leaving potential blind spots in the risk map. The document analysis was limited by restricted access to institutional security records, which lacked the zone-specific and time-specific breakdowns needed for granular historical validation. The weather correlation finding — both suspicious events observed during the study coincided with rainy nights — is based on a one-week window and is insufficient to establish causality or robust correlation. Finally, no implementation or testing has been conducted at this stage; all performance and reliability figures presented in the design are targets, not measured outcomes.

Despite these limitations, the data collected is considered sufficient to guide the design phase. The most important findings — night-time incident concentration, low reporting levels, and preference for a mobile channel — appear independently across interviews, surveys, and direct observation. When three independent methods converge on the same conclusion, confidence in that conclusion is significantly higher than any single method could provide.

7 Methodology

7.1 Systems Analysis Approach (Workshop No. 1)

The systems analysis phase employed a mixed-methods approach [5] that combined six complementary data collection methods to characterize the campus security problem from multiple independent perspectives. Table 2 summarizes the six methods applied.

Table 2. Data Collection Plan — Six Methods Applied to the CSAS

Method	Tool	Sample	Dur.	Objective
Interviews	Structured guide	5 security, 2 admin staff	1 week	Understand current workflows and identify unmet needs.
Surveys	Google Forms	20 students	2 weeks	Assess safety perception, incident experience, and app adoption intent.
Direct observation	Record form	4 campus zones	1 week	Record activity level and suspicious events by time and zone.
Document analysis	Institutional reports	Campus records	3 days	Identify historical incident patterns and cross-validate field findings.
Benchmarking	Comparative analysis	3 apps in Bogotá	1 week	Extract design insights from existing platforms.
Weather monitoring	Weather apps	Teusaquillo locality	1 week	Explore correlation between adverse weather and incident occurrence.

Structured interviews were conducted with five security staff members and two administrative staff members. Three main problems emerged consistently: communication times are slow (the response team may take several minutes to be notified); there is no centralized reporting tool; and staff need faster notifications that include the exact location of incidents, not just verbal descriptions.

Student survey ($n = 20$) measured safety perception, incident experience, incident type, willingness to use a reporting application, and time of day. Seven of the 20 students (35%) reported having experienced or witnessed some type of incident on or around campus. All reported incidents occurred during afternoon and night hours. 100% of students who experienced an incident stated they would use the alert application, compared to 69% among those who had not.

Direct observation was conducted across four campus zones over eight sessions at different times of day. Of the eight sessions, only two recorded suspicious events, and both occurred in the parking lot at night. The parking lot at night was identified as the highest-risk observed zone, with poor lighting and low foot traffic as contributing factors. The suspicious events observed were not reported by any other witness, directly reinforcing the under-reporting hypothesis.

Document analysis of institutional security records confirmed that formal records show far fewer incidents than actually occur according to survey and observation data. The most documented incident types are theft and unauthorized access.

Benchmarking evaluated three existing security alert platforms in Bogotá. Results are presented in Appendix C.

Weather monitoring tracked conditions in the Teusaquillo locality for one week. Both suspicious events coincided with rainy nights, motivating a weather API integration in the system design.

Figure 1 shows the stakeholder relationships and information flows. Figure 2 shows the full system architecture as a node relationship graph from Workshop No. 1.

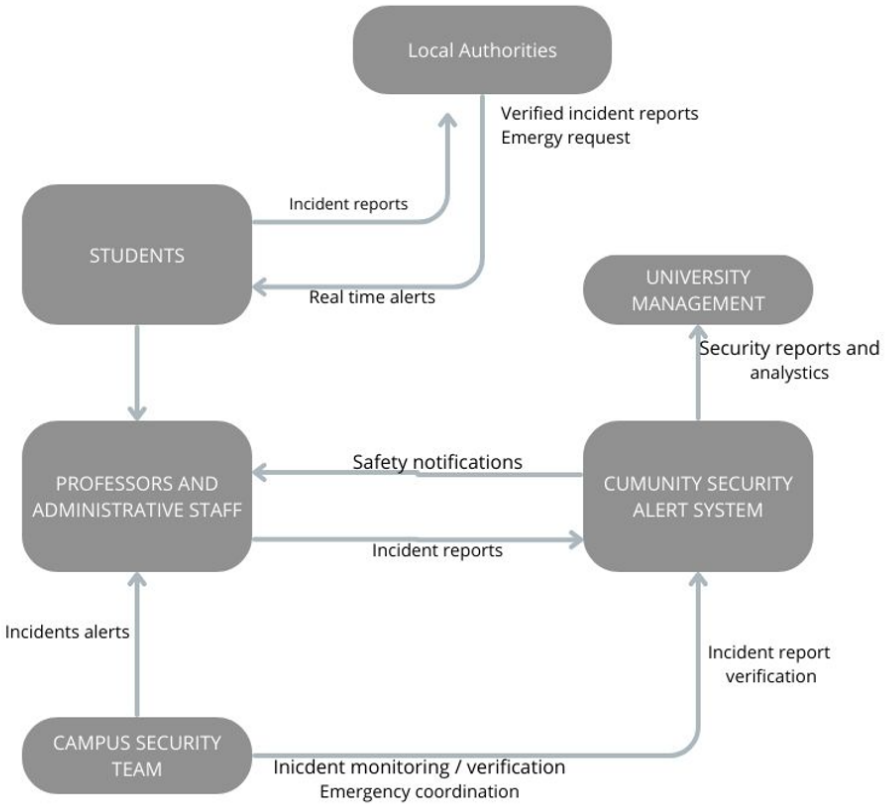

Figure 1. Stakeholder relationships and information flows in the Community Security Alert System (Workshop No. 1).

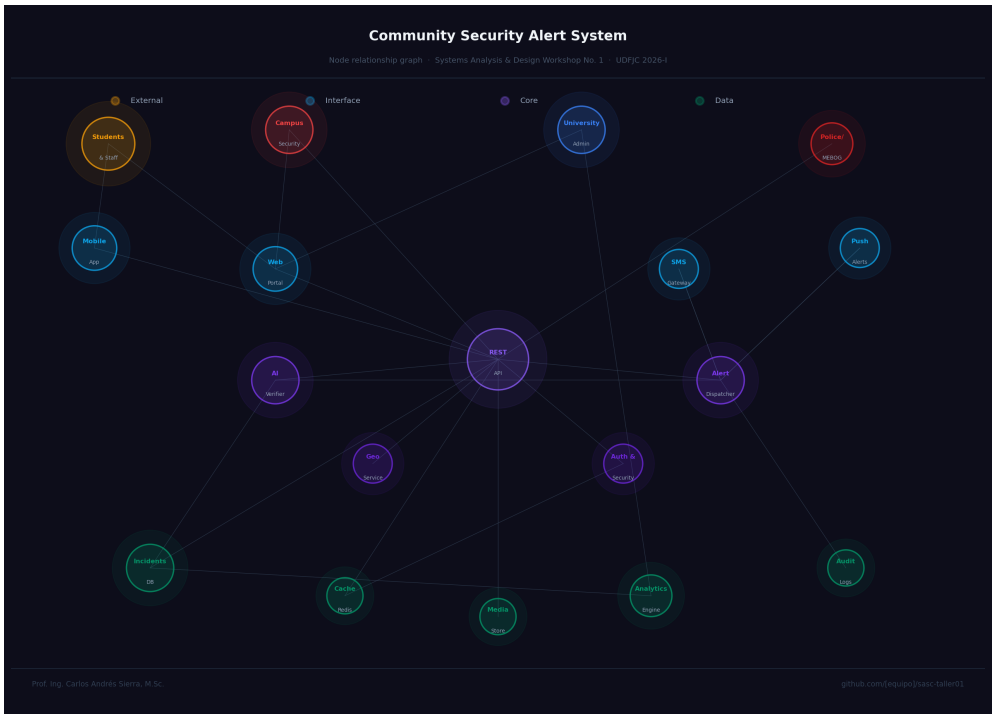

Figure 2. Node relationship graph of the Community Security Alert System. Node size reflects relative centrality; the REST API is the highest-connectivity node (Workshop No. 1).

7.2 System Architecture Design (Workshop No. 2)

The systems design phase translated the empirical findings of Workshop No. 1 into a formal architecture using an iterative, requirements-driven approach consistent with the ISO/IEC/IEEE 15288:2015 systems life cycle [4]. Functional and non-functional requirements were derived directly from identified gaps, with explicit traceability between each requirement and the Workshop No. 1 finding that motivated it. The architectural style was selected based on the non-uniform load profiles of the CSAS subsystems and the need for independent scalability, motivating a microservices-based, event-driven decomposition [8, 7].

The architecture is organized across five layers — Presentation, Application, Service, Data, and Integration. The five core microservices (Incident, Verification, Dispatcher, User, and Analytics) communicate asynchronously through a message broker, ensuring loose coupling between the ingestion and processing stages. Figure 3 shows the architecture diagram. Figure 4 shows the main process flow from incident submission to alert dispatch.

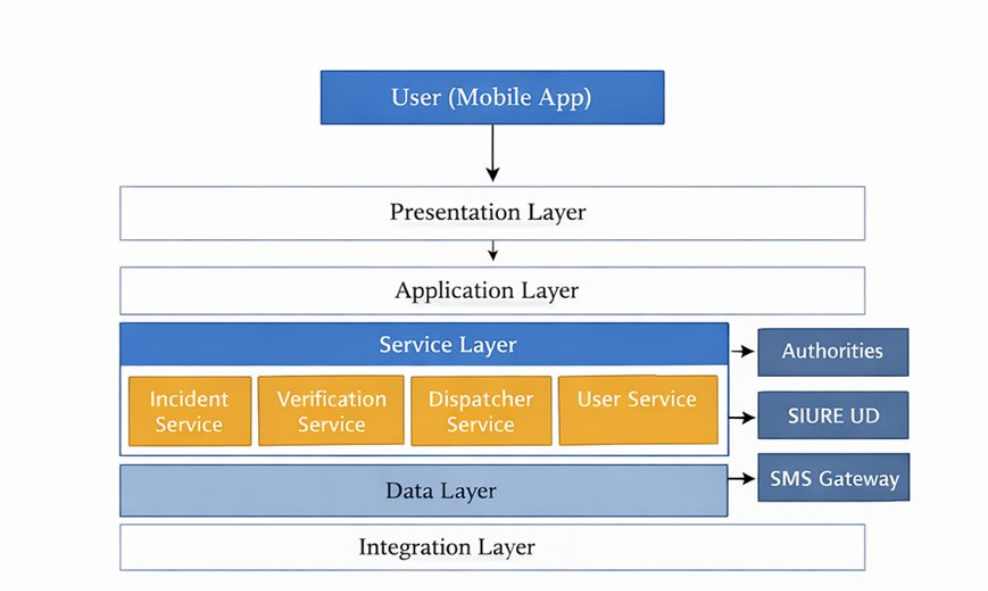

Figure 3. CSAS system architecture organized across five layers. The Service Layer hosts the five core microservices; external integrations (Authorities, SIURE UD, SMS Gateway) connect through the Integration Layer (Workshop No. 2).

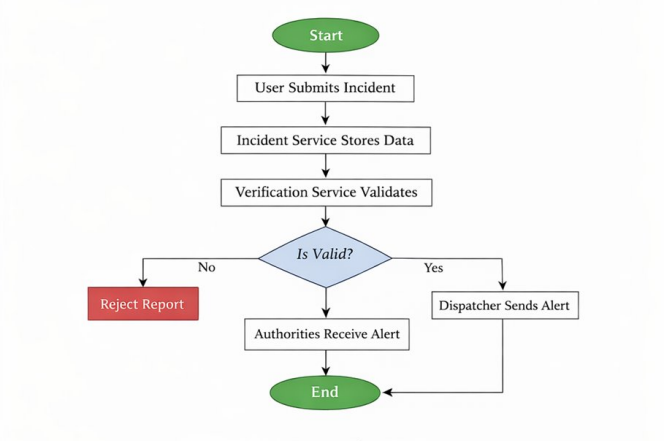

Figure 4. Main process flow diagram. An incident submitted by a user passes through the Verification Service; valid incidents trigger alert dispatch while invalid reports are rejected and logged (Workshop No. 2).

7.3 Technology Stack

Table 3 summarizes the recommended technology stack. Each choice is justified with reference to a specific functional or non-functional requirement.

Table 3. Recommended Technology Stack and Justification

Component	Technology	Justification
Mobile application	React Native	Single codebase for Android/iOS; supports $\leq$3-interaction reporting (NFR-06).
API gateway	Node.js / Nginx	High I/O throughput; handles authentication, rate limiting, and routing.
Backend microservices	Node.js	Asynchronous, event-driven runtime supports the latency target of NFR-01 ($<$30 s).
Verification Engine	Python / FastAPI	Python's ML ecosystem integrates the AI plausibility scorer; FastAPI provides high-performance async endpoints.
Message broker	RabbitMQ	Priority queue support; decouples ingestion from verification to mitigate volume spikes.
Primary database	PostgreSQL + PostGIS	Native geospatial queries for zone-weighted routing and heatmap generation (FR-06, FR-09).
Push notifications	Firebase FCM	Cross-platform delivery; supports the 99.9% delivery target of NFR-02.
SMS fallback	Twilio API	Guaranteed delivery for users without smartphone access; supports equity of access.
Infrastructure	AWS / Azure (K8s)	Container orchestration enables per-service auto-scaling (NFR-03, NFR-05).

8 Requirements

8.1 Functional Requirements

Table 4 presents the ten functional requirements derived directly from Workshop No. 1. Each requirement is explicitly traceable to at least one empirical finding.

Table 4. Functional Requirements and Workshop No. 1 Traceability

ID	Requirement	W1 Rationale
FR-01	Incident reporting via GPS, type, and description	High reporting friction is the primary cause of under-reporting observed across all data collection methods.
FR-02	Geofenced alerts within 500 m of verified incidents	Spatial concentration of incidents in specific zones requires targeted, not broadcast, notifications.
FR-03	AI plausibility scorer for initial validation	Data fragmentation in current manual records increases false positives; automated filtering is needed.
FR-04	Administrative dashboard for security staff	Fragmented phone and walk-up reporting creates the 11.4-minute delay; a centralized view is required.
FR-05	Crowd-sourced confirmation and flagging	100% of incident-experienced survey participants would use the platform, indicating community validation potential.
FR-06	Automatic GPS capture on report submission	Spatial precision requirements identified during four-zone observation sessions.
FR-07	Persistent incident log for pattern analysis	Currently only 41% of events are recorded; historical continuity is absent.
FR-08	One-touch emergency SOS button	Targets a more than 50% reduction from the 11.4-minute baseline response time.
FR-09	Periodic security heatmaps for management	Benchmarking revealed absence of data-driven oversight in comparable platforms.
FR-10	Image and video attachments to reports	Improves evidence quality and verification reliability for ambiguous reports.

8.2 Non-Functional Requirements

Table 5 defines the quality constraints that bound the system's behavior independently of specific functions. Each NFR establishes a measurable threshold against which the implemented system will be evaluated.

Table 5. Non-Functional Requirements and Target Metrics

ID	Category	Requirement
NFR-01	Latency	End-to-end latency from submission to broadcast must be <30 s for 95% of incidents.
NFR-02	Reliability	Notification delivery success $>99.9\%$ across push and SMS channels.
NFR-03	Availability	System uptime $\geq 99.9\%$; guaranteed during high-risk hours (18:00–22:00).
NFR-04	Privacy/Security	All transmissions via TLS 1.3; full compliance with Ley 1581 de 2012 [6].
NFR-05	Scalability	API gateway must sustain a 300% concurrent request surge without performance degradation.
NFR-06	Usability	Incident reporting workflow completable in ≤ 3 interactions.
NFR-07	Maintainability	Any microservice must be updatable or replaceable independently.
NFR-08	Data Integrity	No alert shall broadcast without ≥ 2 community confirmations or 1 administrative approval.

9 Implementation Plan

As this report covers the analysis and design phases only, no implemented system exists at this stage. This section presents the planned implementation strategy, the key performance indicators against which the future system will be evaluated, and the testing approach that will validate the design decisions.

9.1 Implementation Phases

The project follows four sequential phases aligned with the system life cycle.

Phase 1 — Planning and Architecture (Month 1). Finalizes the system architecture, validates all requirements against stakeholder needs, and establishes the development environment. *Success criterion:* all FR and NFR are documented, agreed upon, and traceable to Workshop No. 1 findings. *Deliverable:* architecture diagrams and validated requirements specification.

Phase 2 — Development (Month 2). Implements the frontend (React Native), backend microservices (Node.js, Python), database schema (PostgreSQL + PostGIS), and messaging infrastructure (RabbitMQ). *Success criterion:* an end-to-end prototype in which a user can submit a geolocated incident report (FR-01) and receive a geofenced alert (FR-02). *Deliverable:* functional prototype.

Phase 3 — Testing and Validation (Month 3). Executes the full testing strategy (see Section 9.3). *Success criterion:* system meets NFR-01 (<30 s latency) and NFR-02 (99.9% delivery) under simulated load. *Deliverable:* test reports with pass/fail evidence for all defined test cases.

Phase 4 — Deployment and Monitoring (Month 4). Deploys to the production cloud environment, activates external integrations (SIURE UD, SMS gateway), and establishes live monitoring dashboards. *Success criterion:* 99.9% uptime under real-world conditions. *Deliverable:* deployed system with operational KPI dashboard.

9.2 Key Performance Indicators

Table 6 defines the measurable targets against which the implemented system will be evaluated. Each KPI is aligned with a specific requirement from Section **??**.

Table 6. Key Performance Indicators, Targets, and Measurement Methods

KPI	Target	Measurement	Freq.
Alert latency (end-to-end)	<30 s (NFR-01)	System event logs	Daily
Notification delivery rate	>99.9% (NFR-02)	FCM / Twilio receipts	Daily
System availability	99.9% (NFR-03)	Uptime monitoring	Continuous
Incident recording rate	>85% of events	Cross-ref. staff records	Monthly
Community adoption rate	>60% of students	Monthly active users	Monthly
Verification latency	<60 s	Event broker timestamps	Daily
Alert accuracy rate	≤5% false alarms	Post-incident review	Weekly
Report submission time	≤2 min (NFR-06)	UAT session logs	Per UAT

9.3 Testing Strategy

Unit testing will validate individual components within each microservice (authentication module, incident submission endpoint, notification dispatch function, AI scorer). Each service shall maintain at least 80% code coverage.

Integration testing will verify the complete event chain — Incident Service → Verification Engine → Alert Dispatcher — using a staging environment seeded with campus zone data and simulated user accounts.

Load testing will evaluate system behavior under a 300% traffic surge consistent with NFR-05.

The acceptance criterion is zero timeout errors and end-to-end latency below 30 s at the 95th percentile.

User acceptance testing (UAT) will engage a pilot group of 50 students and 5 security staff members. The primary acceptance criterion is that at least 85% of participants complete their first incident report submission within two minutes.

Security testing will verify TLS 1.3 encryption, authentication mechanisms, access control policies, and compliance with Ley 1581 de 2012 [6], addressing NFR-04.

10 Discussion

10.1 Operational Gap Analysis

The primary motivation for the CSAS is a measurable gap between the current state of campus security and the operational targets the platform is designed to achieve. Table 7 quantifies the most critical dimensions of this gap as established in Workshop No. 1.

Table 7. Operational Gap Analysis: Current State vs. Design Targets

Dimension	Current State	Target	Gap
Emergency response time	$\sim$11.4 min	$<$5 min	$-$6.4 min
Incident recording completeness	$\sim$41%	$>$85%	$+$44 pp
Community app adoption	$\sim$12%	$\geq$60%	$+$48 pp
Alert delivery success rate	$\sim$87% (SMS/call)	$>$99.9%	$+$12.9 pp

These gaps are not exclusively technical. Closing the adoption gap from 12% to 60%, for instance, requires not only a functional mobile application but a sustained community engagement strategy and explicit institutional endorsement from university management — factors that lie at the boundary of the system scope but are critical to real-world effectiveness.

10.2 Risk and Complexity Management

Workshop No. 1 identified five operational parameters with the strongest influence on CSAS behavior. The design incorporates a specific mitigation mechanism for each. A sharp increase in report volume is mitigated by the asynchronous message broker that decouples ingestion from verification, preventing the Verification Engine from becoming a bottleneck. Low user participation is addressed by the low-friction mobile interface (FR-01, $\leq$3 interactions) and the community engagement module planned for Phase 3. Night-time incident concentration is handled by time-weighted priority logic that automatically elevates the urgency of off-hours alerts and activates an on-call escalation path for security staff. High-risk zone clustering — particularly the parking lot, identified as the highest-risk zone across all observation sessions — is managed by zone-based rate limiting and spatial clustering of co-located simultaneous reports.

Adverse weather correlation, while preliminary (based on only two co-occurring observations), will be addressed through real-time weather API integration that adjusts monitoring sensitivity in high-risk zones when conditions deteriorate.

10.3 Feedback Dynamics

The CSAS exhibits two interacting feedback dynamics [3]. The *reinforcing loop* operates as follows: useful alerts increase user vigilance, which increases reporting, which generates more alerts and further improves system effectiveness as adoption grows. The community engagement module targets the critical adoption threshold of approximately 20–25% of the campus population, the point at which report density becomes sufficient to sustain the verification mechanism without heavy dependence on security staff moderation. Below this threshold, the system relies more strongly on staff-initiated verification. The *balancing loop* operates in the opposite direction: excessive alert volume triggers alert fatigue, reducing user engagement and system effectiveness. Two mechanisms work in concert to prevent this degradation: the alert fatigue dampening module in the User Service, which progressively reduces non-critical notification frequency for users with high dismissal rates, and the Verification Engine's quality filters, which prevent low-confidence reports from reaching the dispatcher. Calibrating the confirmation threshold — fast enough for genuine emergencies, strict enough to suppress noise — is the most critical design decision for the implementation phase.

10.4 Ethical Considerations

The CSAS collects three categories of personal data — registration information, real-time location data, and incident media — each subject to distinct obligations under Ley 1581 de 2012, Colombia's general personal data protection regime [6]. The design incorporates four binding obligations: explicit informed consent at registration specific to the safety purpose; purpose limitation (data may not be repurposed or transferred to third parties); user rights to access, correct, and delete personal data; and a defined retention policy (maximum 12 months for incident records before deletion or irreversible anonymization). For sensitive incident types such as harassment or threats, an anonymous reporting option encrypts the reporter's identity at rest, accessible only to system administrators under a documented access-controlled protocol. A formal data processing agreement with the university's legal counsel is a prerequisite to production deployment. Equity of access is addressed through the multi-channel notification architecture: the SMS gateway (NFR-02) ensures that users without smartphones or mobile data access can both receive alerts and submit reports, which is particularly relevant given socioeconomic diversity among the student population.

11 Conclusions

This report has documented the complete analysis and design phases of the Community Security Alert System for Universidad Distrital Francisco José de Caldas. The systems analysis conducted in Workshop No. 1, through six independent data collection methods whose findings consistently

converged on the same three core problems, established an empirical foundation that is robust for a project at this stage. The convergence of interviews, student surveys, direct observation, document analysis, benchmarking, and weather monitoring on the same conclusions — under-reporting, night-time and spatial incident concentration, and the absence of a centralized tool — provides high confidence that the problem characterization is accurate and that design decisions grounded in these findings are well-motivated.

The architecture proposed in Workshop No. 2 directly addresses each identified gap. The low-friction mobile interface and community engagement module target the under-reporting culture. The adaptive zone-weighted and time-weighted dispatch logic of the Alert Dispatcher addresses the non-uniform distribution of incidents. The operational performance gaps are formalized as measurable KPI targets in Table 6 that will enable objective evaluation of the implemented system. Every major design decision is traceable to a specific empirical finding from Workshop No. 1. Five systems engineering principles guided the design throughout: modularity (independent microservices with well-defined interfaces), scalability (per-component auto-scaling), maintainability (containerized CI/CD deployment), reliability (redundancy mechanisms and offline queuing), and privacy (compliance-by-design with Ley 1581 de 2012).

Three open assumptions require validation in the implementation phase: user willingness to report under real operating conditions, verification latency within the 45-second budget, and mobile connectivity coverage across all campus zones. Future work should expand the student survey to a statistically representative sample (minimum $n = 200$, stratified by faculty and time of campus use), conduct a systematic campus connectivity audit before finalizing the notification delivery architecture, and establish formal data-sharing and data processing agreements with SIURE UD and university legal counsel before production deployment.

11 References

[1] C. A. Sierra, *TeamWork as Computer Engineers — CS Collaboration Guidelines*. Bogotá: Universidad Distrital Francisco José de Caldas, 2026.

[2] P. Checkland, *Systems Thinking, Systems Practice*. New York: John Wiley & Sons, 1981.

[3] D. H. Meadows, *Thinking in Systems: A Primer*. White River Junction, VT: Chelsea Green Publishing, 2008.

[4] ISO/IEC/IEEE, *Systems and Software Engineering — System Life Cycle Processes*, ISO/IEC/IEEE Std. 15288:2015, 2015.

[5] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA: SAGE Publications, 2018.

[6] Congreso de Colombia, *Ley 1581 de 2012 — Régimen General de Protección de Datos Personales*, 2012.

[7] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston: Addison-Wesley, 2013.

[8] N. Dragoni *et al.*, "Microservices: Yesterday, Today, and Tomorrow," *Present and Ulterior Software Engineering*, pp. 195–216, 2017.

A Raw Survey Data (Workshop No. 1)

Table 8 presents the complete raw data from the student survey ($n = 20$) conducted during Workshop No. 1. Variables collected were: campus safety perception, incident experience, incident type, willingness to use the alert application, and time of day when incidents occurred.

Table 8. Raw survey data ($n = 20$). Variables: safety perception, incident experience, incident type, app adoption intention, and time of day.

ID	Feels Safe	Experienced	Incident Type	Would Use App	Time
1	No	Yes	Theft	Yes	Night
2	Yes	No	None	Yes	Afternoon
3	No	Yes	Harassment	Yes	Night
4	Yes	No	None	Maybe	Morning
5	Yes	No	None	Maybe	Morning
6	Yes	No	None	Yes	Afternoon
7	No	Yes	Theft	Yes	Night
8	Yes	No	None	Maybe	Morning
9	Yes	No	None	Maybe	Morning
10	Yes	No	None	Yes	Afternoon
11	No	Yes	Suspicious Activity	Yes	Night
12	Yes	No	None	Maybe	Morning
13	No	Yes	Theft	Yes	Evening
14	Yes	No	None	Yes	Afternoon
15	No	Yes	Harassment	Yes	Night
16	Yes	No	None	Maybe	Morning
17	Yes	No	None	Maybe	Night
18	Yes	No	None	Yes	Afternoon
19	No	Yes	Theft	Yes	Night
20	Yes	No	None	Maybe	Morning
Experienced incident:		Yes: 7 (35%)	No: 13 (65%)		

B Direct Observation Record (Workshop No. 1)

Table 9 presents the complete direct observation record from the eight sessions conducted across four campus zones during Workshop No. 1.

Table 9. Direct observation record — 8 sessions across 4 campus zones. Variables: activity level and presence of suspicious event.

Session	Location	Time	Activity Level	Suspicious Event
1	Main Entrance	Morning	High	No
2	Library Area	Afternoon	High	No
3	Parking Lot	Night	Medium	**Yes**
4	Dormitory Area	Evening	Medium	No
5	Main Entrance	Afternoon	High	No
6	Parking Lot	Night	Low	**Yes**
7	Library Area	Morning	Medium	No
8	Dormitory Area	Evening	Medium	No

C Benchmarking of Existing Security Platforms

Table 10 presents the benchmarking comparison of three existing security alert platforms operating in Bogotá conducted during Workshop No. 1.

Table 10. Benchmarking of existing security alert platforms in Bogotá.

Platform	Strengths	Weaknesses	Insight for CSAS
Policía Nacional / Vigila App	Direct police integration; national coverage	Slow response; requires a phone call	Authority integration model is valuable but must be faster and mobile-first.
Neighborhood networks (WhatsApp)	High adoption; instant notifications; familiar channel	No verification; no records; generates panic	Immediacy is a key design goal, but verification is non-negotiable.
SIURE UD (internal)	Exists; staff familiar; institutional endorsement	Email and phone only; no mobile or real-time capability	CSAS complements rather than replaces SIURE UD, building on institutional familiarity.